

Veltrixair.com — Backend

Technical Documentation

Project	Veltrixair.com Backend API
Version	0.0.1
Status	Initial setup — Contact and Careers modules delivered
Runtime target	Node.js 20 LTS or later
Document date	4 August 2026

1. Overview

1.1 Purpose

This service is the backend for the veltrixair.com corporate website. Veltrixair is an AI-led enterprise services firm operating across three jurisdictions — Riyadh (KSA), Dubai (UAE) and Bangalore (India) — delivering custom software, platform engineering, cybersecurity, privacy advisory, managed IT and business applications.

The backend serves the public marketing site and the internal tooling behind it. It is **not** a product API; its traffic profile is anonymous, read-heavy, public-internet traffic with a small number of unauthenticated write endpoints.

1.2 Scope of this release

Module	Status	Covers
Contact	Delivered	Enquiry capture and triage for <code>/contact-us/</code>
Careers	Delivered (Scope A)	Job listings for <code>/careers/</code>
Insights	Delivered (cards only)	Article card grid for <code>/insights/</code>
Discovery	Delivered (Phase 1)	45-minute architect sessions for <code>/talk-to-architect/</code>
Master data	Delivered	Lookup tables shared across modules
Common	Delivered	Cross-cutting services and HTTP primitives
Mail	Delivered (stub transport)	Transactional email behind a swappable seam

Not in this release: identity and RBAC, job applications and résumé upload, insights/blog, newsletter, gated downloads, client portal.

1.3 Design drivers

Three characteristics of the business shape the architecture more than anything else.

Multi-jurisdiction operation. Three offices on three UTC offsets, one of which (Riyadh) works Sunday–Thursday while the other two work Monday–Friday, and one of which (Bangalore) sits on a half-hour offset. Any "respond within N business days" commitment resolves to a different instant per office. This is handled explicitly rather than approximated.

Public regulatory claims. The site publicly states alignment with KSA PDPL, India DPDP 2023, UAE DP Law and GDPR, and displays "*Encrypted in transit · stored in EU/KSA region*" on the contact page. Personal-data handling is therefore implemented as a first-class concern, not retrofitted.

Anonymous public write endpoints. The contact form is an unauthenticated POST on a public site. It will be discovered by scrapers. Abuse resistance is a functional requirement.

2. Technology stack

2.1 Runtime and language

Technology	Version	Purpose
Node.js	24.13.0 (dev)	JavaScript runtime. Target 20 LTS or later in production.
TypeScript	5.9.3	Language. <code>strictNullChecks</code> , <code>noImplicitAny</code> , <code>isolatedModules</code> enabled.
PostgreSQL	17.6	Primary datastore, hosted on Supabase.

2.2 Application framework

Package	Version	Purpose
<code>@nestjs/core</code> <code>@nestjs/common</code>	11.1.28	Application framework — DI container, module system, HTTP pipeline.
<code>@nestjs/platform-express</code>	11.1.28	Express HTTP adapter.
<code>@nestjs/config</code>	4.0.4	Environment configuration with boot-time validation.
<code>@nestjs/swagger</code>	11.4.6	OpenAPI generation, including the CLI plugin that infers schemas from DTOs.
<code>@nestjs/throttler</code>	6.5.0	Rate limiting, applied globally and tightened per route.
<code>rxjs</code>	7.8.2	Observable pipeline used by interceptors.
<code>reflect-metadata</code>	0.2.2	Decorator metadata, required by DI and validation.

Why NestJS. Opinionated module boundaries and first-class dependency injection make the public/admin separation enforceable rather than conventional. The interceptor and filter pipeline gives one place to apply the response envelope and error shape across every route.

2.3 Persistence

Package	Version	Purpose
<code>typeorm</code>	0.3.31	ORM and migration tooling.
<code>@nestjs/typeorm</code>	11.0.3	Nest integration — <code>forRootAsync</code> , repository injection.
<code>pg</code>	8.22.0	PostgreSQL driver.

Why TypeORM over Prisma. Both were evaluated. TypeORM was selected for consistency with the organisation's existing healthcare backend, and because `EntitySubscriber` combined with `AsyncLocalStorage` gives request-

aware audit logging inside the same transaction as the change — a pattern that matters for a firm selling compliance. The trade-off accepted is that TypeORM does not narrow return types on `select`, which is mitigated by `select: false` on personal-data columns (see §9.2).

2.4 Validation and serialisation

Package	Version	Purpose
<code>class-validator</code>	0.15.1	DTO validation via decorators; also validates environment variables at boot.
<code>class-transformer</code>	0.5.1	Payload transformation and type coercion.
<code>dotenv</code>	17.4.2	Loads <code>config/*.env</code> for the standalone TypeORM CLI.

2.5 Tooling

Package	Version	Purpose
<code>@nestjs/cli</code>	11.0.24	Build and scaffolding.
<code>jest</code>	30.4.2	Test runner.
<code>ts-jest</code>	29.4.12	TypeScript transform for Jest.
<code>supertest</code>	7.2.2	HTTP assertions for end-to-end tests.
<code>eslint</code>	9.39.5	Linting, flat config.
<code>typescript-eslint</code>	8.66.0	TypeScript rules, type-checked ruleset.
<code>prettier</code>	3.9.6	Formatting, enforced through ESLint.

2.6 Deliberately absent

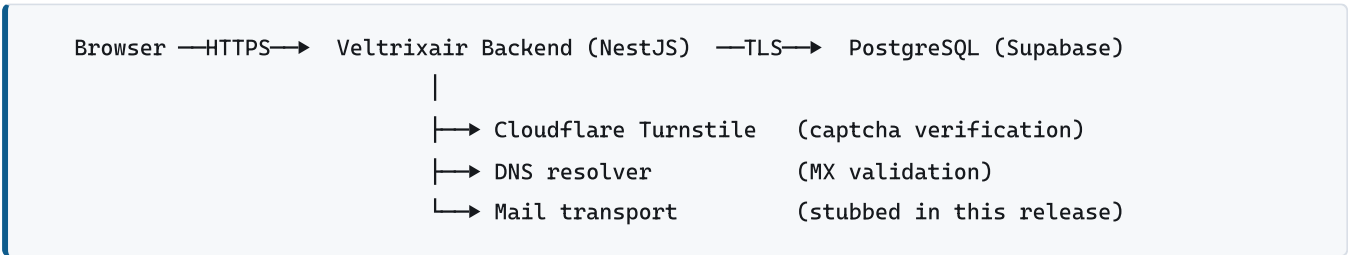
Choices worth recording because their absence is intentional:

Not used	Reason
Redis / BullMQ	Email volume does not yet justify a broker. Mail is dispatched inline after commit. Required before horizontal scaling — see §14.2.
GraphQL	Read patterns are fixed and shallow. REST with generated OpenAPI covers them.
Headless CMS	An admin API is required for leads and jobs regardless; content tables in the same database avoid operating a second system and keep content inside the same residency boundary as personal data.
Date library (Luxon, date-fns)	Business-day arithmetic is implemented against the <code>Intl</code> API, which already carries the IANA timezone database. No dependency, fully unit-tested.
Postgres <code>ENUM</code> types	TypeORM's generated migrations for enum value changes are a known rough edge. Constrained <code>varchar</code> columns with <code>as const</code> unions are used instead.

3. Architecture

3.1 Deployment shape

A single deployable Node process connecting to a managed PostgreSQL instance. Deliberately a modular monolith: traffic is that of a corporate marketing site, and microservice boundaries would add operational cost without addressing any current constraint.



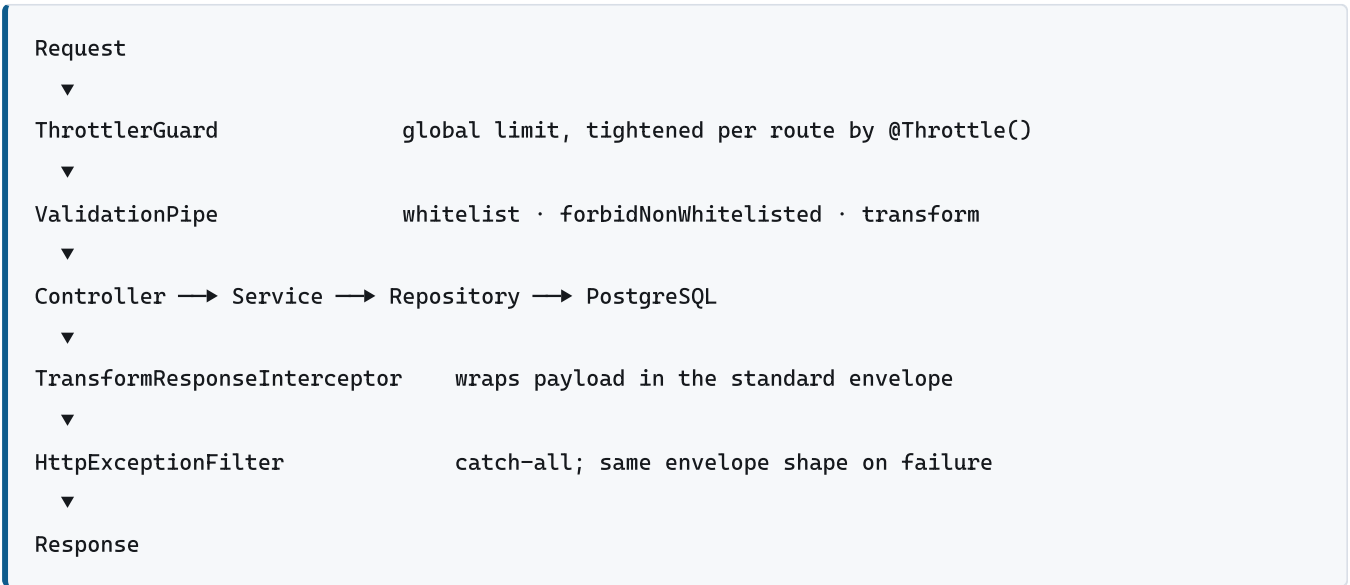
3.2 API surfaces

Routes are separated into two surfaces with different trust assumptions:

Surface	Prefix	Authentication	Rate limit
Public	/contact/... , /careers/...	None	Global, plus per-route limits on writes
Admin	/admin/...	Not yet implemented	Global

Each feature module implements these as **separate controller files** sharing one service, so whether an endpoint is publicly reachable is answerable from the filename.

3.3 Request pipeline



Guards run before pipes, so a throttled request is rejected before validation — meaning a malformed request that is also over the limit returns 429, not 400.

3.4 Module topology

```
AppModule
├─ ConfigModule          @Global · validated at boot
├─ ThrottlerModule       global APP_GUARD
├─ DatabaseModule        TypeORM connection
├─ CommonModule          @Global · ReferenceNumberService, SpamCheckService
├─ MailModule            @Global · MailService
├─ MasterDataModule      @Global · all lookup tables
├─ ContactModule         enquiries
└─ CareersModule         job listings
```

`@Global` is used for genuinely cross-cutting providers only. Feature modules import nothing from each other; anything shared moves into `common`, `mail` or `master-data`.

4. Prerequisites

Requirement	Minimum	Notes
Node.js	20 LTS	Developed on 24.13.0
npm	10	
PostgreSQL	13	Migration uses <code>gen_random_uuid()</code> , core from PG 13. Supabase runs 17.6.

No local PostgreSQL installation is required when using Supabase — the application connects over the network via the `pg` driver.

5. Local setup

```
npm install
cp config/example.env config/dev.env      # then fill in real values
npm run migration:run
npm run start:dev
```

Endpoint	URL
API root	http://localhost:3000
Swagger UI	http://localhost:3000/api/docs
OpenAPI JSON	http://localhost:3000/api/docs-json

There is **no global route prefix**. Endpoints sit at the root: `/contact/enquiries` , `/careers/jobs` .

A Postman collection covering all 24 endpoints ships at `docs/veltrixair.postman_collection.json` .

5.1 npm scripts

Command	Purpose
<code>npm run start:dev</code>	Watch mode
<code>npm run start:prod</code>	Run compiled output from <code>dist/</code>
<code>npm run build</code>	Compile
<code>npm run lint</code>	ESLint with <code>--fix</code>
<code>npm test</code>	Unit tests
<code>npm run test:cov</code>	Coverage
<code>npm run migration:run</code>	Apply pending migrations
<code>npm run migration:revert</code>	Roll back the most recent migration
<code>npm run migration:show</code>	List applied and pending
<code>npm run migration:generate -- src/migrations/Name</code>	Diff entities against the live schema

6. Configuration

6.1 Mechanism

Configuration lives in `config/`, selected by `NODE_ENV`:

- `NODE_ENV=production` → `config/prod.env`
- anything else → `config/dev.env`

Both are gitignored. **`config/example.env` is the only committed copy** and serves as the template.

Every variable is validated at boot by `src/config/env.validation.ts` using `class-validator`. A missing or malformed value **terminates the process at startup** rather than surfacing later as a misleading runtime error. This is deliberate: a container that refuses to start is caught by the deploy health check; one that starts misconfigured becomes a production incident.

6.2 Reference

Variable	Required	Description
<code>NODE_ENV</code>	Yes	<code>development</code> <code>production</code> <code>test</code>
<code>PORT</code>	Yes	HTTP port, default 3000
<code>DB_HOST</code>	Yes	Database host
<code>DB_PORT</code>	Yes	Database port — use 5432
<code>DB_USERNAME</code>	Yes	Database user
<code>DB_PASSWORD</code>	Yes	Database password
<code>DB_NAME</code>	Yes	Database name. On Supabase this is literally <code>postgres</code>
<code>DB_SSL</code>	No	Auto-enabled for <code>*.rds.amazonaws.com</code> ; set <code>true</code> for Supabase
<code>CORS_ORIGINS</code>	Yes	Comma-separated allowlist of browser origins
<code>THROTTLE_TTL</code>	No	Rate-limit window in ms, default 60000
<code>THROTTLE_LIMIT</code>	No	Requests per window per IP, default 100
<code>PRIVACY_NOTICE_VERSION</code>	Yes	Recorded verbatim against every consent
<code>IP_PEPPER</code>	Yes	Minimum 32 characters. HMAC key for hashing submitter IPs
<code>MAIL_FROM</code>	Yes	Sender address
<code>TURNSTILE_SECRET</code>	No	When unset, captcha verification is skipped

6.3 IP_PEPPER — operational note

IPv4 has roughly four billion values, so an attacker holding the database but not the pepper cannot reverse the stored hashes; an attacker who can guess the pepper can enumerate every address instantly. The pepper is therefore the only control making those hashes irreversible.

Generate a distinct value per environment:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Never rotate it once live. Changing the pepper makes every previously stored `ip_hash` unmatchable. Treat it as un-rotatable key material and back it up accordingly.

6.4 Supabase connectivity

Use the **direct connection on port 5432**, not the transaction pooler on 6543. The transaction pooler does not preserve session state, which breaks DDL and rejects the `-c timezone=UTC` startup option this application sends. If a pooler is required, use the session pooler and note that the username becomes `postgres.<project-ref>`.

7. Project structure

config/	Environment files by NODE_ENV
docs/	Postman collection, this document
src/	
main.ts	Bootstrap: pipes, interceptor, filter, CORS, Swagger
app.module.ts	Root module
data-source.ts	TypeORM CLI entrypoint
config/	
env.validation.ts	Boot-time environment schema
database/	
database.config.ts	Shared connection options
database.module.ts	TypeOrmModule.forRootAsync
migrations/	Hand-written, timestamp-ordered
common/	@Global – cross-cutting
decorators/	@ResponseMessage
dto/	PaginationQueryDto, PaginatedResult
filters/	HttpExceptionFilter
interceptors/	TransformResponseInterceptor
services/	ReferenceNumberService, SpamCheckService
utils/	business-hours.util (+ spec)
mail/	@Global – MailService
master-data/	@Global – lookup tables
entities/	7 master entities
master-data.service.ts	
contact/	Enquiries
careers/	Job listings

7.1 Feature module layout

src/contact/	
contact.module.ts	
contact.service.ts	Single service, both surfaces
enquiry-routing.service.ts	Module-specific provider at module root
public-contact.controller.ts	Anonymous
admin-contact.controller.ts	Staff
dto/	
entities/	

`entities/` holds `@Entity()` classes only; providers sit at the module root. A `services/` subfolder is introduced only when a module has three or more providers.

8. Data model

Twelve tables, excluding TypeORM's `migrations` bookkeeping table.

8.1 Master tables

Table	Rows	Purpose
<code>office_masters</code>	3	Riyadh, Dubai, Bangalore — timezone and working days
<code>country_masters</code>	11	Contact form jurisdictions; maps country → owning office
<code>industry_masters</code>	9	Industry dropdown
<code>enquiry_topic_masters</code>	7	Enquiry topic; maps topic → destination inbox
<code>enquiry_timeline_masters</code>	5	Project timeline dropdown
<code>practice_area_masters</code>	8	Careers practice filter
<code>job_location_masters</code>	3	Careers location filter — cities only

Masters use an **integer code as the natural key**; foreign keys reference the code column rather than the UUID primary key. Codes begin at **101** in every table and increment independently. Both conventions are inherited from the organisation's existing backend.

`office_masters` is not presentation data. It carries `timezone`, `working_days`, `work_start_hour` and `work_end_hour`, and is the input to every SLA calculation in the system.

8.2 Contact module

`contact_enquiries` — one row per submission.

Group	Columns
Identity	<code>reference_no</code> (unique, VLX-YYYY-NNNNNN)
Form fields	<code>topic_code</code> , <code>full_name</code> , <code>company</code> , <code>role_title</code> , <code>work_email</code> , <code>phone</code> , <code>country_code</code> , <code>industry_code</code> , <code>timeline_code</code> , <code>message</code> , <code>requires_nda</code>
Consent	<code>consent_at</code> , <code>privacy_notice_version</code>
Routing / SLA	<code>office_code</code> , <code>routed_to_email</code> , <code>sla_due_at</code> , <code>first_responded_at</code> , <code>status</code> , <code>assigned_to</code>
Provenance	<code>source_page</code> , <code>utm_source</code> , <code>utm_medium</code> , <code>utm_campaign</code> , <code>ip_hash</code> , <code>user_agent</code> , <code>spam_score</code>
Housekeeping	<code>is_deleted</code> , <code>deleted_at</code> , <code>created_date</code> , <code>updated_date</code>

`status` is constrained to NEW · QUALIFYING · ENGAGED · WON · LOST · SPAM.

contact_enquiry_events — append-only timeline. Event types: `CREATED`, `STATUS_CHANGED`, `ASSIGNED`, `NOTE_ADDED`, `MESSAGE_VIEWED`, `NOTIFICATION_SENT`. `MESSAGE_VIEWED` exists specifically to record reads of NDA-flagged enquiries.

contact_enquiry_ref_seq — PostgreSQL sequence backing reference numbers. A sequence rather than `COUNT(*) + 1`, which races under concurrent submissions and produces duplicates.

8.3 Careers module

job_postings — one row per role, seeded with the 14 currently listed (R-001 – R-014). Carries `ref_code`, `slug`, `classification` (`practice_code`, `office_code`, `work_mode`), `terms` (`employment_type`, `experience_label`, `experience_min_years`, `experience_max_years`, `visa_sponsorship`), `presentation` (`hot_role`, `display_order`, SEO fields) and `lifecycle` (`status`, `posted_at`, `closes_at`).

job_posting_locations — join table. A role may span several locations ("Dubai / Riyadh", "Bangalore / Remote"), so location filtering uses an `EXISTS` subquery rather than a filtered join, ensuring a single-location filter does not truncate the location list returned per row.

8.4 Indexing

Beyond primary and unique keys:

- Foreign-key columns on `contact_enquiries` and `job_postings`
 - `contact_enquiries` (`created_date` DESC) — default admin ordering
 - Partial index `contact_enquiries` (`sla_due_at`) WHERE `first_responded_at` IS NULL AND `is_deleted` = `false` — drives the overdue filter
 - Partial index `job_postings` (`hot_role` DESC, `display_order` ASC) WHERE `status` = 'OPEN' AND `is_deleted` = `false` — drives the default public listing
-

9. Cross-cutting behaviour

9.1 Response envelope

Every response, success or failure, uses one shape:

```
{
  "success": true,
  "statusCode": 201,
  "message": "Enquiry received",
  "data": { },
  "timestamp": "2026-08-04T10:15:43.696Z",
  "path": "/contact/enquiries",
  "method": "POST"
}
```

`message` is set per handler with `@ResponseMessage('...')`. Paginated endpoints return `{ items, total, page, limit, totalPages }` in `data`.

Errors use the same envelope with `success: false`. Validation failures place the field errors in `data` with `message: "Validation failed"`. Any other structured detail a thrower attaches is preserved in `data` too — a blackout that clashes with a confirmed session returns the clashing bookings in `data.conflicts`, so the caller can act on it rather than only knowing that something failed.

9.2 Personal-data containment

Columns holding personal data are declared `select: false`:

Table	Columns
<code>contact_enquiries</code>	<code>message</code> , <code>phone</code> , <code>ip_hash</code>

TypeORM excludes these unless explicitly requested with `addSelect()`. This inverts the default from *leak unless the developer is careful to safe unless deliberately requested* — the closest available approximation to compile-time protection, given TypeORM does not narrow types on `select`.

The admin list endpoint additionally uses an explicit column projection. Both controls are verified by end-to-end assertions.

9.3 Business-day arithmetic

`src/common/utils/business-hours.util.ts` computes SLA due dates against an office's working calendar.

Office	Working week	Offset
Riyadh	Sunday–Thursday	UTC+3
Dubai	Monday–Friday	UTC+4
Bangalore	Monday–Friday	UTC+5:30

An enquiry arriving Thursday 17:00 in Riyadh is due **Sunday**; the same instant in Dubai is due **Friday**. Implemented with `Intl.DateTimeFormat` — no date library — and covered by 15 unit tests including the half-hour India offset and a ten-business-day span.

9.4 Abuse resistance

Applied to public write endpoints:

Control	Weight	Decisive alone
Honeypot field filled	100	Yes
Turnstile verification failed	100	Yes
Disposable email domain	60	Yes
No MX record for domain	30	No

Threshold is 50. `no_mx_record` is deliberately weighted below it: DNS resolution fails on transient resolver problems as well as genuinely mail-less domains, and a false positive must never lose a legitimate enquiry.

Submissions scoring at or above the threshold are **stored with status: SPAM, never rejected**. The caller receives an ordinary success response — telling a bot it was detected lets it tune around the filter, and a mistakenly flagged prospect must not see a failure. A false positive is recovered by changing status, at which point the SLA clock starts normally.

Rate limits: 100 requests/minute/IP globally; 5 requests/hour/IP on `POST /contact/enquiries`.

Limitation. Throttler state is in-process. Running more than one instance multiplies the effective limit; shared storage (Redis) is required before horizontal scaling.

9.5 Confidential enquiries

The contact form offers *"This enquiry includes confidential information. Please send a mutual NDA before we go further."* When set:

- The message body is **omitted** from notification email; the notice links to the admin record instead
- Any admin read of the full record writes a `MESSAGE_VIEWED` event naming the actor

10. API reference

10.1 Public

Method	Route	Description
GET	/contact/form-options	All four contact dropdowns in one response
POST	/contact/enquiries	Submit an enquiry — 5/hour/IP
GET	/careers/filters	Practice, location (cities) and work-mode filter chips
GET	/careers/jobs	List open roles — <code>practice</code> , <code>location</code> , <code>workMode</code> , <code>search</code> , <code>hotOnly</code> , <code>sort</code> , <code>page</code> , <code>limit</code>
GET	/careers/jobs/:slug	Role detail
GET	/insights/filters	Type (with badge colours), topic and region chips
GET	/insights/articles	Card grid — <code>type</code> , <code>topic</code> , <code>region</code> , <code>search</code> , <code>sort</code> , <code>page</code> , <code>limit</code>
GET	/discovery/practices	Six practices with their architects
GET	/discovery/availability	Per-day density — requires <code>timezone</code>
GET	/discovery/slots	Bookable times for one visitor-local date
POST	/discovery/bookings	5/hour/IP — atomic slot claim, 409 if taken
GET	/discovery/bookings/:token · POST /:token/cancel	Attendee self-service

Discovery admin — architects and their calendars:

Method	Route	Notes
GET POST	/admin/discovery/architects · /:id	
PATCH	/admin/discovery/architects/:id	Name, credentials, practice, office
PATCH	/admin/discovery/architects/:id/practice	Reassign — the calendar follows the architect
DELETE	/admin/discovery/architects/:id	409 while upcoming sessions exist
GET PUT	/admin/discovery/architects/:id/availability	Weekly pattern; a day off is an absent weekday
GET POST	/admin/discovery/blackouts	409 if the period covers a confirmed session
DELETE	/admin/discovery/blackouts/:id	Reopens the slots it closed

10.2 Admin

These routes are not authenticated in this release. See §15.

Method	Route	Description
GET	/admin/contact/enquiries	Filter by <code>status</code> , <code>topicCode</code> , <code>officeCode</code> , <code>search</code> , <code>overdue</code>
GET	/admin/contact/enquiries/:id	Full record including message body
GET	/admin/contact/enquiries/:id/events	Timeline
PATCH	/admin/contact/enquiries/:id/status	First move into a working status stops the SLA clock
PATCH	/admin/contact/enquiries/:id/assign	
POST	/admin/contact/enquiries/:id/notes	
GET	/admin/careers/jobs	Includes drafts and closed roles
GET	/admin/careers/jobs/:id	
POST	/admin/careers/jobs	
PATCH	/admin/careers/jobs/:id	
PATCH	/admin/careers/jobs/:id/status	DRAFT OPEN CLOSED
DELETE	/admin/careers/jobs/:id	Soft delete
GET	/admin/insights/articles/:id	Includes drafts and archived
POST	/admin/insights/articles	
PATCH	/admin/insights/articles/:id/ /status	
DELETE	/admin/insights/articles/:id	Soft delete

11. Database and migrations

`synchronize` is **disabled in every environment**. Schema changes occur only through migrations, applied as a discrete pre-deploy step — never on application boot.

11.1 Applied migrations

Timestamp	Name	Creates
1785801600000	CreateContactModule	5 masters, <code>contact_enquiries</code> , <code>contact_enquiry_events</code> , reference sequence
1785805200000	CreateCareersModule	2 masters, <code>job_postings</code> , <code>job_posting_locations</code>
1785808800000	RenumberMasterCodesFrom101	Shifts every master code and its references by +100
1785812400000	SeparateRemoteFromLocations	Removes "Remote" from the location taxonomy; remote-ness moves to <code>work_mode</code>
1785816000000	CreateInsightsModule	3 masters, <code>articles</code> , <code>article_regions</code> ; seeds the 14 cards
1785819600000	RemoveArticleFeaturedFlag	Drops <code>featured</code> — hero selection is a frontend concern
1785823200000	CreateDiscoveryModule	Practices, architects, availability rules, blackouts, slots, bookings

11.2 Standards

Migrations are hand-written. `migration:generate` diffs decorators against a live database and produces noisy — occasionally destructive — SQL. Anything touching personal-data columns is authored deliberately.

An applied migration is immutable. TypeORM tracks migrations by timestamp, so editing a file changes nothing on a database that has already run it; the result is silent divergence between environments. Corrections are issued as new migrations.

Verification. Run `migration:generate` twice in succession; the second run must produce an empty migration. If it does not, the entities and the database genuinely disagree.

Seeds. Prefer separate schema and seed migrations for new work, following the existing organisational convention.

12. Conventions

12.1 Inherited

Aligned with the organisation's existing backend so both codebases read the same way:

- Tables named `snake_case_plural`; columns explicitly mapped with `@Column({ name: '...' })`
- Constraints named `vtx_<table>_<column>_pk|fk|unique`; indexes `idx_<table>_<column>`
- Master tables keyed on an integer code starting at 101; foreign keys reference the code column
- Housekeeping columns `is_active`, `is_deleted`, `deleted_at`, `created_date`, `updated_date`
- A single `buildBaseOptions()` feeding both the Nest module and the CLI `DataSource`
- Standard response envelope and `@ResponseMessage()` decorator
- Environment files in `config/`, selected by `NODE_ENV`

12.2 Deliberate deviations

Aspect	Existing backend	This service	Rationale
Timestamps	<code>timestamp</code> , session TZ <code>Asia/Kolkata</code>	<code>timestamptz</code> , session TZ UTC	Three offices on three offsets; no single local timezone is correct for all
CORS	<code>origin: true</code> (reflects any origin)	Explicit allowlist	Public site with credentialed requests; reflecting any origin defeats the protection
Environment	Trusted, silent fallbacks	Validated at boot	Multiple environments drift silently
Rate limiting	None	Global plus per-route	Anonymous public write endpoints
Enumerations	Postgres <code>ENUM</code> types	<code>varchar + as const</code> union + <code>CHECK</code>	TypeORM enum-change migrations are unreliable; adding a value becomes a one-line constraint change

Each deviation reflects a difference in traffic profile: the existing backend is an internal, single-country, fully authenticated system. This one is public, multi-region and largely anonymous.

13. Testing

```
npm test          # unit
npm run test:cov   # coverage
```

Unit tests — 15, all passing. Concentrated on `business-hours.util`: the SLA arithmetic is the component most likely to be subtly wrong, is pure, and is therefore cheap to test exhaustively. Coverage includes each office's working week, the half-hour India offset, weekend roll-over in both calendars, and multi-day spans.

End-to-end verification. Both modules were exercised against the live database:

Module	Assertions	Covers
Contact	26	Form options, consent enforcement, reference format, per-office SLA divergence, topic and country routing, honeypot handling, throttle, PII containment, NDA audit, status transitions
Careers	24	Filter options, all filter dimensions, combined filters, keyword search, multi-location integrity, slug detail, 404 handling, pagination, admin visibility

`package.json` points `test:e2e` at `./test/jest-e2e.json`, which does not yet exist — specs are colocated under `src/`. Harmless until that script is run.

14. Build and deployment

14.1 Build

```
npm ci
npm run build          # emits dist/
npm run migration:run  # discrete pre-deploy step
npm run start:prod
```

14.2 Production readiness

Required before this service handles production traffic:

Item	Status
Admin authentication	Blocking — see §15
Real mail transport	Required for notifications to be delivered
<code>TURNSTILE_SECRET</code> configured	Recommended; honeypot and MX checks operate without it
Distinct <code>IP_PEPPER</code> per environment	Required
<code>CORS_ORIGINS</code> set to production origins	Required
Shared throttler storage (Redis)	Required before running more than one instance
Structured logging and error reporting	Not yet configured
Health and readiness endpoints	Not yet implemented
Container image and CI pipeline	Not yet created

14.3 Operational notes

- Sequences do not roll back. Deleting test enquiries leaves gaps in reference numbering; this is cosmetic.
 - TypeORM issues `CREATE EXTENSION IF NOT EXISTS "uuid-oss" on connect`. Harmless on Supabase, but the production role's privileges are worth reviewing.
 - The application logs SQL when `NODE_ENV=development`. Query parameters contain personal data and must not be logged in production; the current configuration already restricts production logging to errors and warnings.
-

15. Known gaps

Gap	Severity	Detail
Admin routes unauthenticated	Blocking	<code>/admin/**</code> exposes lead personal data without authentication. Guard insertion points are marked at class level in both admin controllers, and <code>actor</code> is already threaded through every mutating call so the audit trail is correct as soon as real authentication is introduced. Must not be deployed in this state.
Mail transport stubbed	High	<code>MailService.dispatch()</code> logs rather than sends. Replacing it requires no caller changes.
Turnstile inactive	Medium	Skipped unless <code>TURNSTILE_SECRET</code> is set.
Careers Scope B	Planned	Applications, résumé upload, virus scanning, hiring pipeline. Requires an object-storage decision.
Async job processing	Planned	Email dispatch is inline after commit.
Remaining site modules	Planned	Insights, newsletter, gated downloads, client portal.

15.1 Open question

The careers page states two different response commitments — "*within five business days*" for listed roles and "*within ten business days*" for senior applications. This should be resolved with the content owner before Careers Scope B, as it determines the SLA constant.

Appendix A — Reference codes

Master codes begin at **101** in every table and increment independently, matching the convention used by the organisation's existing backend.

Enquiry topics → destination inbox

Code	Topic	Inbox
101	New Engagement	contact@
102	RFP / RFI	contact@
103	Partnership	contact@
104	Voice AI Demo	voiceai@
105	Privacy Advisory	privacy@
106	Managed Services	contact@
107	Other	contact@

Countries → owning office

Code	Country	Office
101	Saudi Arabia (KSA)	Riyadh
102	United Arab Emirates	Dubai
103–106	Bahrain, Qatar, Kuwait, Oman	Riyadh
107	India	Bangalore
108–111	United Kingdom, European Union, United States, Other	Riyadh

Industries — 101 Banking & Finance · 102 Government & Public Sector · 103 Energy & Utilities · 104 Healthcare & Life Sciences · 105 Real Estate & Hospitality · 106 Retail & E-Commerce · 107 Manufacturing & Logistics · 108 Education & EdTech · 109 Other

Timelines — 101 Immediate (this quarter) · 102 Next 1–3 months · 103 3–6 months · 104 6–12 months · 105 Exploratory / Researching

Practice areas — 101 Software · 102 Platform · 103 Cyber · 104 Privacy · 105 Managed · 106 Business Apps · 107 Growth · 108 Ops

Article types — 101 Insight · 102 Case Study · 103 Whitepaper · 104 Regulatory Update · 105 Perspective (each carries a badge colour)

Article topics — 101 Software · 102 Platform · 103 Cybersecurity · 104 Data Privacy · 105 AI & ML · 106 Business Apps · 107 Growth

Regions — 101 KSA · 102 GCC · 103 India · 104 Global (editorial regions, distinct from `country_masters`)

Job locations — 101 Riyadh · 102 Dubai · 103 Bangalore

Cities only. "Remote" is a working arrangement, not a place, and lives on `job_postings.work_mode` .

Work modes — `ONSITE` · `HYBRID` · `REMOTE` (a `CHECK` -constrained column, not a master table)

Offices

Code	Office	Timezone	Working week
101	Riyadh HQ	Asia/Riyadh	Sunday–Thursday
102	Dubai Regional Office	Asia/Dubai	Monday–Friday
103	Bangalore Delivery Centre	Asia/Kolkata	Monday–Friday